

Scalable Logic Gate Sizing using ML Techniques and GPU Acceleration

Bing-Yue Wu¹, Rongjian Liang², Geraldo Pradipta², Anthony Agnesina²,
Haoxing Ren², and Vidya A. Chhabria¹

¹Arizona State University ²NVIDIA Research

0. Version History

02/01/2024: Initial draft of the problem statement sent for review

02/15/2024: Feedback from the review. Minor changes. Need for upper bound on runtime?

1. Introduction

With scaling and the slowdown in Moore's law, EDA tools must work increasingly harder to achieve power, performance, and area (PPA) specifications. A critical step for PPA improvement is timing optimization, which, today, is heuristic-based and may not result in optimal solutions. An integral part of timing optimization is the NP-hard logic gate sizing problem, which involves selecting a size for every netlist instance from a set of choices in the standard cell library, each with a different delay/area/power. The goal is to assign gate sizes to each instance to minimize the area/power of the circuit while satisfying delay constraints. The problem is challenging due to (i) the non-convexity of the delay models, (ii) the discrete space of gate sizes, (iii) the number of near-critical paths, (iv) the tradeoff between power and timing, and (iv) the resource availability constraints which prevent the exploration of the complete design space. Logic gate sizing has been studied for over three decades now. In early work, the gate sizes were assumed to be continuous, and the timing models were either derived from RC Elmore delay models, which can be transformed into a convex function of sizes or approximated by a convex function [1]. Later, for discrete gate sizing, several heuristics emerged that used sensitivity-based methods [2], convex programming [3], and Lagrangian relaxation (LR)-based [4], [5] methods. LR-based methods have found tremendous success, but under realistic nonconvex timing models, they use heuristics to choose the order of gate sizing and work over a small local neighborhood of the circuit [4]. In the recent past, machine learning techniques have been leveraged to speed up LR [6], reinforcement learning (RL) has been applied to address the gate sizing problem [7], and generative AI-based techniques have been used for netlist optimization [8]. However, ML-LR is still based on heuristics, and RL techniques are not scalable.

The contest aims to overcome these challenges of gate sizing techniques and drive academic research in timing optimization. In this context, the contest serves three purposes:

- 1) Lower the barrier to entry for non-EDA experts by converting traditional EDA problems to an ML-solvable problem through ML-friendly data representation formats.
- 2) Explore the state-of-the-art algorithms for gate sizing to drive academic research to generate scalable gate-sizing algorithms using GPU acceleration or machine learning (supervised, unsupervised, or reinforcement learning-based techniques).
- 3) Release updated benchmarks in a FinFET technology node with evaluation scripts to accelerate research in logic gate sizing.

Contest goal: The goal of the contest is to develop a scalable sizer that solves a constrained optimization problem, which is formulated as:

$$\text{minimize } \sum_{i \in I} \text{Power}_{c_i}$$

$$\text{subject to } \text{slack}(pin_i) \geq 0; \text{slew}(pin_i) \leq k_1; \text{load}(pin_i) \leq k_2 \quad \forall i \in I$$

$$c_i \in C_i \quad \forall i \in I$$

where I is a set of instances; C_i is the set of choices for instance i in the library; $c_i \in C_i$ is the library cell assigned to instance i ; Power_{c_i} is the power of instance i ; $\text{slack}(pin_i)$ and is slack at all pins of instance i ; $\text{slew}(pin_i)$ and is the transition time at all pins of instance i , $\text{load}(pin_i)$ is the load capacitance seen at all pins of instance i , and k_1 , and k_2 are constants. The goal is to minimize the generated solution's power (switching and leakage) while meeting slack, max slew, and max load constraints.

2. Input/Output Files and Formats

Input Formats

The contest will provide inputs in two formats. The first is standard EDA files, and the second is CircuitOps' intermediate data representation format. Both formats are explained below:

(i) Standard EDA file formats:

- .v file: Gate-level Verilog netlist
- .def file: Post-placement connections and cell locations
- .sdc file: Constraint file provided in OpenROAD's TCL to specify input port delay and transition, output port capacitance, and specified clock period
- .lib file: Library file which consists of the look-up-tables for delay and output slew

If participants choose to work with these files, they are expected to work with them as input and develop parsers. If participants choose to develop their sizer in C/C++ they can leverage [OpenROAD](#) [9] and [OpenSTA](#) (submodule in OpenROAD) [10] header files for parsing liberty and netlist. Suppose participants choose to develop their sizer in Python. In that case, they can use OpenROAD Python APIs to query information they need, or they can reuse existing Python packages such as [liberty-parser](#) [11] and [verilog-parser](#) [12]. Participants can also work with their own parsers if they cannot extract the information they need from existing APIs. We will provide example APIs in scripts for querying information from OpenROAD DB if you choose to use OpenROAD as the parser.

(ii) CircuitOps data representation format:

The CircuitOps data representation format provides an ML-friendly format consisting of a graph

and annotated features. The graph is a labeled property graph that is backed by relational tables.

The graph utilizes a node for a net, pin, and cell of a circuit. It has four types of edges to represent connections between pin to pin nodes (pin-pin), pin to cell nodes (pin-cell), pin to net nodes (pin-net), and cell to net nodes (cell-net). Each node and edge has associated relational tables with properties associated with the node and edge. The relational tables will be provided to the contestants, which can easily be parsed into their size by using standard CSV readers in C++/Python. Details on the list of properties and their description and types of nodes and edges can be found in the Appendix, the [CircuitOps paper](#) [13], and the [GitHub repository](#) [14]. **These files will be in a CSV format.**

The choice of which input format is left to the participants. The first EDA file-based format will provide you with maximum information to develop your gate sizer and the ability to extract custom features for ML models. The second will be limited to the features and properties available in CircuitOps, but it will allow you to skip the tedious parsing tasks and focus on the ML/GPU acceleration aspects. We will release translators from standard EDA file format to CircuitOps format (the translators will leverage OpenROAD APIs and database queries).

Intermediate Files

These files will provide timing, power, and effective capacitance information from the OpenROAD timer. The timing information (.timing) will be stored in a file with the format below:

```
<pinName or portName> <riseSlack> <fallSlack> <riseTransition>
<fallTransition> <riseArrival> <fallArrival>
```

The power information will be stored in a file (.power) with the format below:

```
<instanceName> <leakagePower> <switchingPower>
```

The effective capacitance for each drive pin will be stored in a file (.ceff) with the format below:

```
<pinName> <Ceff>
```

Output File Format

The output file (.size) should have the following format:

```
<instanceName> <libCellName>
```

3. Benchmarks

We will release benchmarks all in the ASAP7 technology node. The benchmarks will vary in size from 10K-1M instances. We will release ten visible benchmarks and will have ten hidden benchmarks that we will evaluate your solutions on. For the ten visible benchmarks, we will also

provide reasonably good quality (.size) output files for the ability to use supervised learning-based machine learning techniques.

4. Developing the Gate Sizer: Rules, assumptions, timer, and support scripts.

Assumptions: The contest will focus on optimizing setup time only. The sizing of sequential gates will not be permitted. All sequential gates in the benchmark will be rising edge triggered. The benchmarks will assume an ideal clock. There will be no clock buffers in the circuit (zero skew). The contest will be setup on post-placement benchmarks. We will provide effective capacitance at the driver pin for output gate delay and slew computations.

Timing Engine: You can implement your own timer and power estimator within your sizer. However, the final evaluation will use OpenROAD's timer (OpenSTA). You may also use OpenROAD's timer and power estimator as part of your sizer. We will provide examples of Python APIs interacting with the OpenROAD timer. The APIs will be in Python and white-boxed. If you are developing your code in C++, you can utilize the corresponding C++ APIs that the Python APIs wrap around within OpenROAD.

Support Scripts: We will provide two types of support scripts:

(i) It will use standard EDA tool files as an example and operate with OpenROAD Python APIs to update gate sizes and obtain updated timing and power numbers. We will provide scripts that operate with this format with either File I/Os (input: standard EDA files and .sizes file and outputs: timing and .power files) or directly with function calls returning standard Python data structures (input: python list of sizes and output: python list for timing and power values).

(ii) It will use CircuitOps as input, update gate sizes using OpenROAD Python APIs, and provide updated timing and power numbers, and update the CircuitOps format with either File I/Os (input: relational tables CSVs and output: table CSV) or directly with pandas data frames as inputs and outputs.

6. Evaluation: Metrics, Platforms, and Scripts

Metrics: The sizer will be evaluated on the quality of the generated solution and runtime. The following cost function will be used to score the submissions:

$$Score = (1 + \delta * runTimeF) (Power_{tot} + \alpha * worstSlack + \beta * worstSlew + \gamma * maxLoad)$$

where $\alpha = \begin{cases} 0 & \text{if } worstSlack \geq 0, \\ \text{else } slackPenalty \end{cases}$

$$\beta = \begin{cases} 0 & \text{if } worstSlew \leq k_1, \\ \text{else } slewPenalty \end{cases}$$

$$\beta = \begin{cases} 0 & \text{if } worstSlew \leq k_2, \\ \text{else } loadPenalty \end{cases}$$

$$runTimeF = \log_2(runTime/medianRunTime)$$

The *runTime* is the runtime of the benchmark being evaluated, and the *medianRunTime* is the median of the runtimes of all teams submission's on the benchmark being evaluated. The values of k_1 and k_2 will be provided for the technology node. The values of the penalty terms will be decided later. Each participating team will be given a score for each benchmark. The scores will be added across all testcases, and the top three teams with the least scores will win the contest.

Platform: The submitted sizers will run on a computing platform equipped with four A100 GPU, which has a memory capacity of 80GB. The utilization of up to 8 CPU threads will be supported. We will supply a Docker image preconfigured with CUDA and some popular ML libraries to facilitate a standardized development environment. We encourage participating teams to capitalize on the potential of GPU acceleration and explore the integration of ML techniques. However, the usage of the GPU is optional, and teams are free to choose their preferred approach, whether it involves leveraging the GPU or not.

Scripts: We will release our evaluation scripts so teams can test their scores on visible benchmarks.

References

- [1] J. P. Fishburn and A. E. Dunlop, "TILOS: A Posynomial Programming Approach to Transistor Sizing," in Proc. ICCAD, 1985.
- [2] J. Hu, et al., "Sensitivity-Guided Metaheuristics for Accurate Discrete Gate Sizing," in Proc. ICCAD, 2012.
- [3] K. Kasamsetty, et al., "A New Class of Convex Functions for Delay Modeling and its Application to the Transistor Sizing Problem," IEEE T. Comput. Aid. D., vol. 19, no. 7, pp. 779–788, 2000.
- [4] A. Sharma, et al., "Fast Lagrangian Relaxation-Based Multithreaded Gate Sizing Using Simple Timing Calibrations," IEEE T. Comput. Aid D., vol. 39, no. 7, pp. 1456–1469, 2020.
- [5] C.-P. Chen, et al., "Fast and Exact Simultaneous Gate and Wire Sizing by Lagrangian Relaxation," IEEE T. Comput. Aid D., vol. 18, no. 7, pp. 1014–1025, 1999.
- [6] X. Zhou et al., "Heterogeneous Graph Neural Network-based Imitation Learning for Gate Sizing Acceleration," in Proc. ICCAD 2022,
- [7] Y. -C. Lu, S. Nath, V. Khandelwal and S. K. Lim, "RL-Sizer: VLSI Gate Sizing for Timing Optimization using Deep Reinforcement Learning," in Proc DAC, 2022
- [8] S. Nath, G. Pradipta, C. Hu, T. Yang, B. Khailany and H. Ren, "TransSizer: A Novel Transformer-Based Fast Gate Sizer," in Proc. ICCAD, 2022.
- [9] <https://github.com/The-OpenROAD-Project/OpenROAD>
- [10] <https://github.com/The-OpenROAD-Project/OpenSTA>
- [11] <https://pypi.org/project/liberty-parser/>
- [12] <https://pypi.org/project/verilog-parser/>
- [13] R. Liang, A. Agnesina, G. Pradipta, V. A. Chhabria and H. Ren, "Invited Paper: CircuitOps: An ML Infrastructure Enabling Generative AI for VLSI Circuit Optimization," in Proc. ICCAD 2023.

[14] <https://github.com/NVlabs/CircuitOps>

Appendix

CircuitOps format description:

Cell properties:

Cell_name: name of the cell

Is_seq: whether the cell is sequential (register, flip flop, etc.)

Is_macro: whether it is a macro or a standard cell

Is_in_clk: whether the cell is in clock net

X0, y0, x1, y1: the smallest x-coordinate, the smallest y-coordinate, the largest x-coordinate, and the largest y-coordinate of the cell bounding box

Is_buf, is_inv: whether the cell is a buffer or and inverter

Libcell_name: the libcell name of the cell

Cell_static_power, cell_dynamic power: the static and dynamic power of the cell at the current timing corner

Libcell properties:

Libcell_name: name of the library cell

Func_id: The functional ID of a libcell is a unique identifier assigned based on its functionality.

Libcells that perform the same function will share the same functional ID, while those with different functionalities will have distinct IDs.

Libcell_area: area of the libcell

Worst_input_cap: the largest input capacitance

Libcell_leakage: leakage power of the libcell

Fo4_delay: gate delay considering a load capacitance four times larger than the largest input capacitance, while the input slew remains a predefined constant

Libcell_delay_fixed_load: gate delay considering a predefined constant load capacitance and a predefined constant input slew

Net properties:

Net_name: name of the net

Net_route_length: total routing path length of the net

Net_steiner_length: total length of the steiner tree of the net

Fanout: fanout of the net

Total_cap: total capacitance of the net, including wire capacitance and the sink pin capacitance

Net_cap: total wire capacitance

Net_coupling: coupling capacitance

Net_res: wire resistance

Pin properties:

Pin_name: name of the pin

X, y: coordinates of the center of the pin

Is_startpoint: whether it is a timing path starting point

Is_endpoint: whether it is a timing path ending point
Dir: direction of the pin, i.e., whether it is an input pin or output pin
Maxcap: capacitance constraint of the pin
Maxtran: transition time constraint of the pin
Num_reachable_endpoint: count of reachable endpoints
Cell_name: name of the corresponding cell
Net_name: name of the corresponding net
Pin_tran: transition time of the pin
Pin_slack: slack of the pin
Pin_rise_arr: arrival time of the pin at rising edge
Pin_fall_arr: arrival time of the pin at falling edge
Input_pin_cap: input pin capacitance

Pin_pin_edge:

Src: driving pin name
Tar: sink pin name
Is_net: whether the pin2pin edge is a net connection
Arc_delay: timing arc delay from the src to tar